2026-10-12 MySQL操作库操作

- 1. MySQL 数据库入门与库操作
  - 1.1 为什么需要数据库
  - 1.2 主流数据库
- 2. MySQL 服务、数据库与表
  - 2.1 启动与管理 MySQL 服务
  - 2.2 连接 MySQL 服务器
  - 2.3 服务器、数据库和表的逻辑关系
  - 2.4 最小使用案例
- 3. MySQL 架构、SQL 分类与存储引擎
  - 3.1 MySQL 架构
  - 3.2 SQL 分类
  - 3.3 存储引擎
- 4. 数据库字符集与校验规则
  - 4.1 字符集与校验规则分别解决什么问题
  - 4.2 继承关系与“就近原则”
  - 4.3 查看当前编码配置
  - 4.4 校验规则会影响查询结果
  - 4.5 utf8 与 utf8mb4
- 5. 创建数据库
  - 5.1 基本语法
  - 5.2 创建示例
  - 5.3 创建后得到什么
- 6. 查看和切换数据库
  - 6.1 查看所有数据库
  - 6.2 查看数据库定义
  - 6.3 选择当前数据库
- 7. 修改数据库属性
  - 7.1 修改默认字符集和校验规则
  - 7.2 修改数据库默认值不会自动转换旧表
  - 7.3 MySQL 没有常规的数据库重命名语句
- 8. 删除数据库
  - 8.1 删除语法
  - 8.2 删除数据库的后果
- 9. 数据库备份与恢复
  - 9.1 备份保存的是当前状态

- 9.2 备份一个数据库
- 9.3 恢复数据库
- 10. 综合实操与检查清单
  - 10.1 从零创建并验证数据库
  - 10.2 操作逻辑总结
  - 10.3 常见错误

1. MySQL 数据库入门与库操作

> 本章从“为什么需要数据库”开始，逐步建立 **MySQL 服务端 → 数据库 → 表 → 行/列** 的逻辑模型，再系统整理数据库的创建、查看、切换、修改、删除和备份恢复。数据库是多个表的逻辑集合，表结构及表中数据的操作将在下一章展开。

1.1 为什么需要数据库

如果只用普通文件保存程序数据，通常会遇到以下问题：

- 文件的安全性和访问控制不完善。
- 文件不利于大量数据的查询和管理。
- 文件不利于存储海量数据。
- 文件在程序中的控制不方便。
  数据库的数据可以存放在磁盘或内存中，但它不仅能保存数据，还负责组织数据、控制访问、支持查询和管理海量数据。**数据库的设计和使用水平，是衡量程序员能力的重要指标之一。**

  > 数据库不是“很多文件简单地放在一起”，而是一套有结构、有规则、由数据库服务统一管理的数据组织方案。

1.2 主流数据库

数据库	特点与典型场景
SQL Server	微软产品，常与 .NET 技术栈配合，适合中大型项目
Oracle	甲骨文产品，适合大型项目和复杂业务逻辑；课件认为其并发通常不如 MySQL
MySQL	属于甲骨文，Web 应用中使用广泛，并发性好，常用于电商、SNS、论坛等场景，对简单 SQL 的处理效果较好
PostgreSQL	由加州大学伯克利分校计算机系开发，开源、免费，可修改和分发，适合通用关系型数据管理

数据库	特点与典型场景
SQLite	轻量级关系型数据库，遵守 ACID，常嵌入应用程序或设备，资源占用低
H2	Java 开发的嵌入式数据库，本身可作为类库嵌入应用项目

后续以 MySQL 为主要操作对象。

2. MySQL 服务、数据库与表

2.1 启动与管理 MySQL 服务

安装 MySQL 时，安装的是数据库管理系统及其后台服务。课件展示了不同系统上的安装方式，例如 CentOS 和 Windows 下的 MySQL 安装；实际学习时以当前机器已安装的版本为准。
在 Windows 中可以通过以下方式查看服务状态：

1. 按 `Win + R`，输入 `services.msc`。
2. 找到 MySQL 服务。
3. 使用左侧按钮启动、停止、暂停或重新启动服务。

> `mysqld` 是 MySQL 服务端进程；`mysql` 是用于连接并操作数据库的客户端程序，两者作用不同。

2.2 连接 MySQL 服务器

使用的连接命令如下：

```
mysql -h 127.0.0.1 -P 3306 -u root -p
```

各选项含义：

- `-h`：主机地址。省略时默认连接本地主机。
- `-P`：端口号，MySQL 默认端口是 `3306`。注意这里是大写 `P`。
- `-u`：用户名。
- `-p`：提示输入密码。通常应让程序交互式询问密码，避免把密码直接写在命令中。
  连接成功后，客户端会显示连接编号、服务端版本，并进入 `mysql>` 提示符。SQL 语句

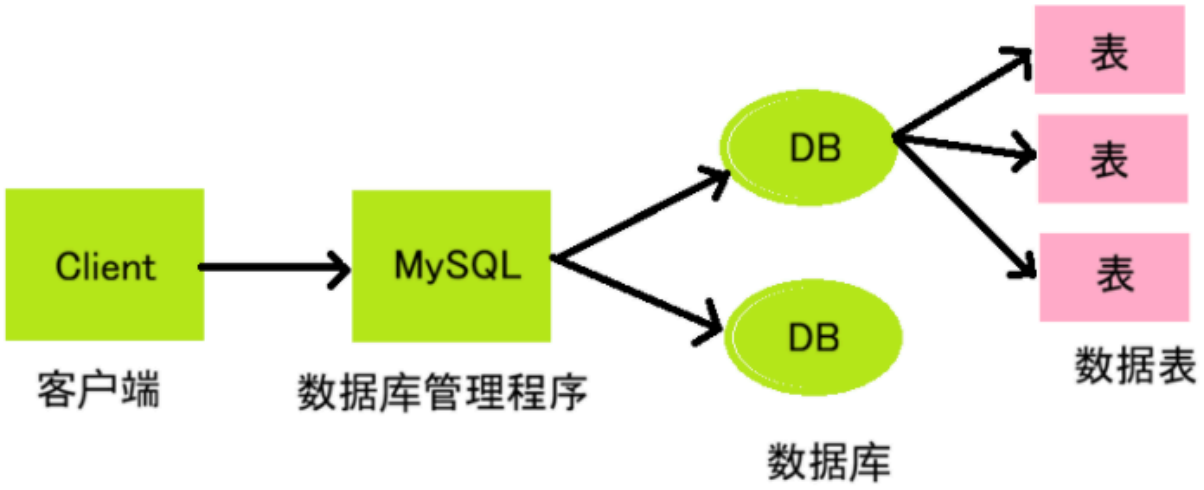
通常以分号 `;` 结束。
也可以使用更简短的本地连接方式：

```
mysql -u root -p
```

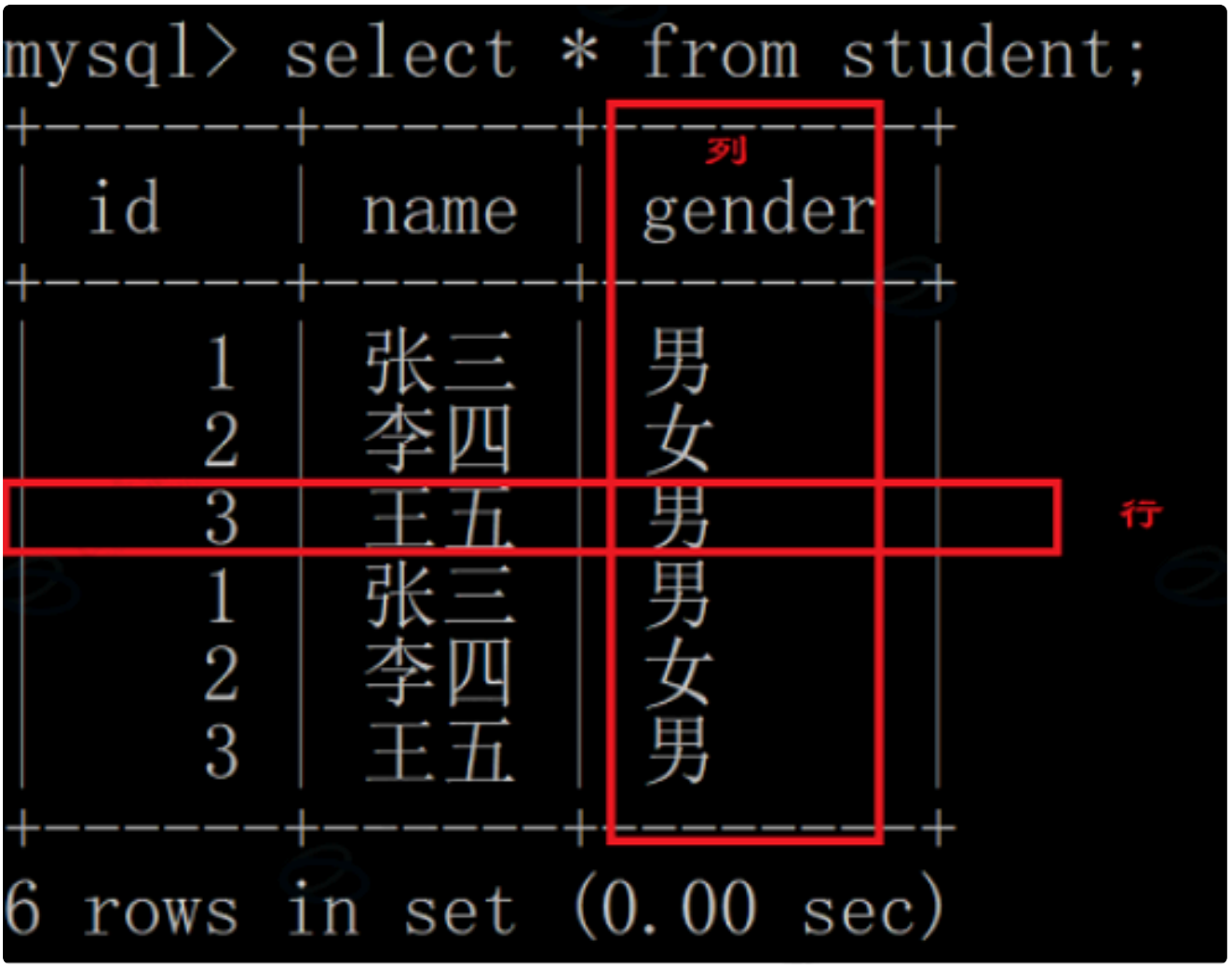
2.3 服务器、数据库和表的逻辑关系

正确理解以下三层对象，是学习“库操作”的前提：

1. **数据库服务器**：一台机器上安装的数据库管理系统，可以管理多个数据库。
2. **数据库**：面向一个应用或一组业务数据建立的逻辑命名空间，可以包含多张表、视图、触发器等对象。
3. **表**：保存某一类实体数据的结构化对象。表由列定义结构，由行保存具体记录。



客户端的请求先发送给 MySQL 服务器，再由服务器访问指定数据库中的表。一个数据库可以有多张表，一个表可以有若干行和列。



> 表依赖于数据库。未指定数据库时，直接创建或访问表会因为没有当前数据库而失败。

2.4 最小使用案例

用一个学生表展示MySQL 的基本使用流程：

```
-- 1. 创建数据库
CREATE DATABASE helloworld;

-- 2. 选择数据库
USE helloworld;

-- 3. 创建表
CREATE TABLE student (
    id INT,
    name VARCHAR(32),
    gender VARCHAR(2)
);

-- 4. 插入数据
```

```sql
INSERT INTO student (id, name, gender) VALUES (1, '张三', '男');
INSERT INTO student (id, name, gender) VALUES (2, '李四', '女');
INSERT INTO student (id, name, gender) VALUES (3, '王五', '男');

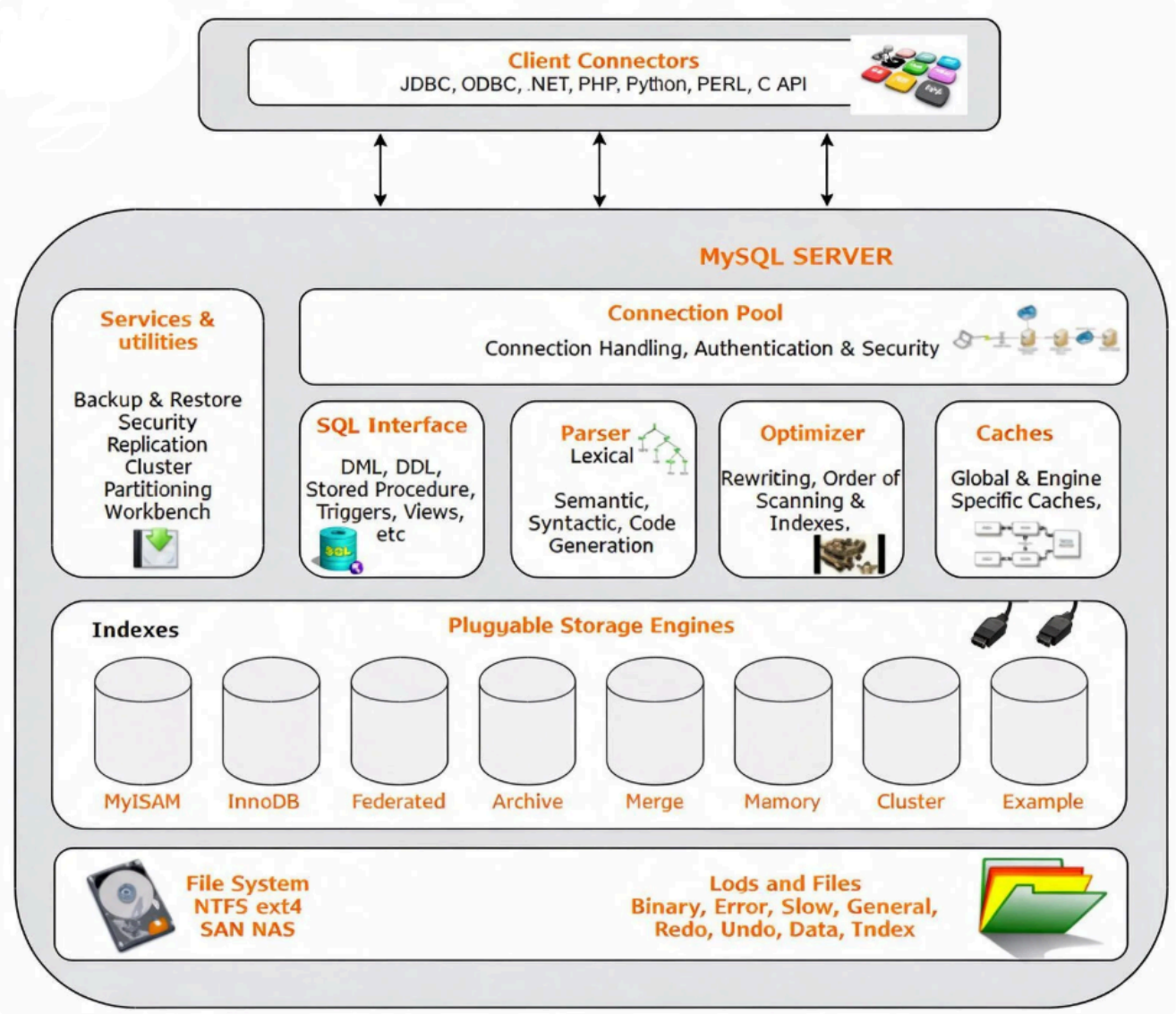
-- 5. 查询数据
SELECT * FROM student;
```

其中 `*` 表示查询表中的所有列。查询结果中的一条记录就是一行，一列则表示一类数据项。

3. MySQL 架构、SQL 分类与存储引擎

3.1 MySQL 架构

MySQL 可以在 Unix/Linux、Windows、macOS 等系统上运行。不同平台的底层实现不同，但 MySQL 基本保持一致的逻辑结构。



从外到内可以概括为：

1. **客户端连接层**：JDBC、ODBC、.NET、PHP、Python、C API 等客户端发起连接请求。

2. **MySQL Server 核心层**：连接池负责认证与安全；SQL 接口接收 DDL、DML 等语句；解析器执行词法和语法分析；优化器选择执行方案；缓存组件辅助查询。
3. **可插拔存储引擎层**：InnoDB、MyISAM、Memory 等组件负责数据的实际存取、索引和事务能力。
4. **文件系统层**：数据、索引及日志最终保存到磁盘上的文件系统，并可能涉及 Redo Log、Undo Log 等日志。

> 数据库的增删改查由 MySQL 服务统一执行，底层仍然需要在文件系统中读写数据，但程序员不需要直接操作这些文件。

3.2 SQL 分类

分类	英文名称	作用	代表指令
DDL	Data Definition Language	定义和维护数据库、表等存储结构	`CREATE`、`DROP`、`ALTER`
DML	Data Manipulation Language	操作表中的数据	`INSERT`、`DELETE`、`UPDATE`
DQL	Data Query Language	查询数据，通常作为 DML 的独立分类	`SELECT`
DCL	Data Control Language	管理权限和事务控制	`GRANT`、`REVOKE`、`COMMIT`

本章的数据库创建、修改和删除属于 DDL；`SHOW`、`USE` 等语句属于数据库管理与元数据操作。SQL 关键字大小写不敏感，但统一使用大写更易阅读。

3.3 存储引擎

存储引擎规定了数据库管理系统如何存储数据、如何建立索引，以及如何更新和查询数据。MySQL 的核心特点是**插件式存储引擎**，不同表可以选择不同的引擎。
查看当前 MySQL 支持的存储引擎：

```
SHOW ENGINES;
```

```
mysql> show engines;
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| Engine             | Support | Comment                                                        | Transactions | XA   | Savepoints |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| InnoDB             | DEFAULT | Supports transactions, row-level locking, and foreign keys     | YES          | YES  | YES        |
| MRG_MYISAM         | YES     | Collection of identical MyISAM tables                          | NO           | NO   | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables      | NO           | NO   | NO         |
| BLACKHOLE          | YES     | /dev/null storage engine (anything you write to it disappears) | NO           | NO   | NO         |
| MyISAM             | YES     | MyISAM storage engine                                          | NO           | NO   | NO         |
| CSV                | YES     | CSV storage engine                                             | NO           | NO   | NO         |
| ARCHIVE            | YES     | Archive storage engine                                         | NO           | NO   | NO         |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                             | NO           | NO   | NO         |
| FEDERATED          | NO      | Federated MySQL storage engine                                 | NULL         | NULL | NULL       |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
9 rows in set (0.00 sec)
```

现代 MySQL 通常以 InnoDB 作为默认存储引擎。课件还对常见引擎作了横向比较；下表中的“是/否”表示课件所列能力，“版本相关”表示实际结果取决于 MySQL 的具体版本。

特性	MyISAM	BDB	Memory	InnoDB	Archive	NDB
存储限制	无	无	有	64TB	无	有
事务	否	是	否	是	否	是
锁定粒度	表级	页级	表级	行级	行级	行级
MVCC / 快照读	否	否	否	是	否	是
地理空间数据	是	否	否	是	是	是
B 树索引	是	是	是	是	否	否
哈希索引	否	否	是	否	否	否
全文检索索引	是	否	否	是	否	否
聚簇索引	否	否	否	是	否	否
数据缓存	否	否	是	是	否	是
索引缓存	是	否	是	是	否	是
数据压缩	是	否	否	是	是	否
通过函数加密数据	是	是	是	是	是	是
存储空间成本	低	低	不适用	高	很低	低
内存成本	低	低	中等	高	低	高
批量插入速度	高	高	高	低	很高	高
集群支持	否	否	否	否	否	是
复制支持	是	是	是	是	是	是
外键	否	否	否	是	否	否
备份 / 时间点恢复	是	是	是	是	是	是
查询缓存	是	是	是	是	是	是
数据字典统计信息更新	是	是	是	是	是	是

需要区分一个容易混淆的点：**存储引擎是表的属性，不是数据库的属性。** 创建数据库时只设置数据库级默认字符集和校验规则，并不会为整个数据库指定一个存储引擎；存储引擎要在后面的 `CREATE TABLE` 中设置。

将 `COMMIT` 归入 DCL；不少教材会把事务控制语句另分为 TCL（Transaction Control Language）。分类名称不同不影响实际语句的使用，但复习时应了解这一术语差异。

4. 数据库字符集与校验规则

4.1 字符集与校验规则分别解决什么问题

创建数据库时会涉及两个相关但不同的概念：

1. **字符集（character set）**：规定字符如何编码成字节，决定数据库能够存储哪些字符。
2. **校验规则（collation）**：规定字符如何比较和排序，影响等值判断、范围判断及排序结果。
   例如，是否把 `A` 和 `a` 视为相同、重音不同的字母是否等价，由校验规则决定，而不只是由字符集决定。

> 插入数据的本质仍然是“存取”：客户端输入字符，经过连接编码传送；数据库按照列、表、数据库的字符集保存；查询时再把结果返回客户端。只要其中某个环节声明错误，就可能出现乱码、比较结果异常或字符无法保存。

4.2 继承关系与“就近原则”

字符集和校验规则通常按照以下层级继承：

```
MySQL 服务端默认值
        ↓
数据库默认值
        ↓
表默认值
        ↓
列默认值
```

更具体的显式设置会覆盖上层默认值。因此，课件笔记中把它概括为“就近原则”：

- 创建数据库时没有写明字符集，就采用服务端默认值。
- 创建表时没有写明字符集，就采用所在数据库的默认值。
- 创建列时没有单独设置，就采用表的默认值。
  表依赖于数据库，因此数据库的默认字符集和校验规则会影响后续表及列，但不会反过来自动修改已经创建的对象。

4.3 查看当前编码配置

查看服务端默认值：

```
SHOW VARIABLES LIKE 'character_set_server';
SHOW VARIABLES LIKE 'collation_server';
```

进入某个数据库后查看数据库默认值：

```
SHOW VARIABLES LIKE 'character_set_database';
SHOW VARIABLES LIKE 'collation_database';
```

查看当前连接正在使用的输入、连接和结果字符集：

```
SELECT
    @@character_set_client,
    @@character_set_connection,
    @@character_set_results;
```

客户端与服务器之间的编码如果不匹配，可以使用 `SET NAMES` 统一当前会话的编码：

```
SET NAMES utf8mb4;
```

这一步设置的是**当前连接会话**，重新连接后需要重新确认；数据库和表的默认字符集则属于持久化对象设置。

4.4 校验规则会影响查询结果

以不区分大小写的 `utf8mb4_unicode_ci` 和按二进制比较的 `utf8mb4_bin` 为例：

```
SELECT 'A' = 'a' COLLATE utf8mb4_unicode_ci AS ci_result;
SELECT 'A' = 'a' COLLATE utf8mb4_bin       AS bin_result;
```

- `_ci` 通常表示大小写不敏感（case-insensitive）。
- `_cs` 通常表示大小写敏感（case-sensitive）。

- `_bin` 按二进制值比较。
  如果查询条件中的字符串使用了 `_ci` 规则，`WHERE name = 'A'` 可能同时匹配 `A` 和 `a`。因此，**插入的数据相同，不代表在不同校验规则下的查询和排序结果也相同。**
  比较两个列时还要注意校验规则是否兼容。不同校验规则直接比较可能报 `Illegal mix of collations`，应明确统一列定义或显式使用 `COLLATE`。

4.5 `utf8` 与 `utf8mb4`

课件中的示例使用 `utf8`，在旧版 MySQL 中它通常最多使用 3 个字节表示一个字符，不能完整覆盖所有 Unicode 字符，例如部分 Emoji 和扩展汉字。现代项目新建数据库时通常优先使用 `utf8mb4`。
兼容性较广的推荐示例是：

```
CREATE DATABASE school
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_unicode_ci;
```

如果服务器明确支持 MySQL 8.0 的排序规则，也可以根据业务需要选择 `utf8mb4_0900_ai_ci` 等规则。不要直接照搬未知版本中的校验规则名称。

5. 创建数据库

5.1 基本语法

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
    [DEFAULT] CHARACTER SET [=] charset_name
    [DEFAULT] COLLATE [=] collation_name;
```

说明：

- MySQL 中 `DATABASE` 和 `SCHEMA` 基本是同义词。
- `IF NOT EXISTS` 可以避免数据库已存在时直接报错。
- `CHARACTER SET` 也可以简写为 `CHARSET`。
- `DEFAULT` 可以省略。
- 真正的数据库名、字符集和校验规则名称不需要加引号；如果标识符与关键字冲突，应使用反引号。

5.2 创建示例

最小示例：

```
CREATE DATABASE helloworld;
```

该语句没有显式指定字符集和校验规则，因此采用 MySQL 服务端的默认配置。
指定字符集的示例：

```
CREATE DATABASE IF NOT EXISTS school
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

仅指定字符集时，MySQL 会选择与该字符集兼容的默认校验规则。为了便于定位查询行为，也可以把字符集和校验规则同时写清楚。

5.3 创建后得到什么

创建数据库相当于在 MySQL 中建立一个逻辑命名空间，之后可以在其中创建表、视图、函数、触发器等对象。底层数据最终仍由 MySQL 服务写入其数据目录，但不应该绕过 MySQL 直接修改这些目录和文件。
需要特别记住：

- 创建数据库时没有 `ENGINE` 选项。
- 存储引擎在后续创建表时指定。
- 数据库名在大小写敏感的文件系统上可能区分大小写，为避免移植问题，不应仅靠大小写区分两个数据库名。
- 创建数据库通常需要相应的 `CREATE` 权限。

6. 查看和切换数据库

6.1 查看所有数据库

```
SHOW DATABASES;
```

也可以用模式匹配筛选：

```
SHOW DATABASES LIKE 'sch%';
```

MySQL 通常还会显示系统数据库，例如：

- `information_schema`：提供元数据视图。
- `mysql`：保存用户、权限等系统信息。
- `performance_schema`：性能监控相关数据。
- `sys`：基于性能和元数据视图提供更易读的诊断信息。
  这些系统库通常不应作为普通业务库直接修改。

6.2 查看数据库定义

```
SHOW CREATE DATABASE school;
```

结果会显示数据库的默认字符集和校验规则。它可以用于确认创建结果，也便于迁移时复用同样的定义。

6.3 选择当前数据库

```
USE school;
```

执行后，当前连接会话的默认数据库切换为 `school`，后面的表操作可以省略数据库名前缀。
查看当前选择的数据库：

```
SELECT DATABASE();
```

如果尚未选择数据库，该函数返回 `NULL`。`USE` 的作用范围是当前客户端连接；重新建立连接后，需要再次选择数据库，或者用 `数据库名.表名` 的形式访问对象。

7. 修改数据库属性

7.1 修改默认字符集和校验规则

```
ALTER {DATABASE | SCHEMA} db_name
    [DEFAULT] CHARACTER SET [=] charset_name
```

```
    [DEFAULT] COLLATE [=] collation_name;
```

示例：

```
ALTER DATABASE school
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

字符集和校验规则必须相互兼容，否则 MySQL 会拒绝修改。

7.2 修改数据库默认值不会自动转换旧表

这是数据库操作中非常容易出错的地方：

> `ALTER DATABASE` 修改的是数据库今后的默认设置，不会自动把已有表及其列转换为新字符集。

已有表需要使用表级转换，例如：

```
ALTER TABLE student
    CONVERT TO CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

表转换会重写数据，可能耗时、占用空间并影响锁和业务可用性。生产中应先备份、在测试环境验证，并评估停机窗口。表结构和表数据的完整操作留到下一章展开。

7.3 MySQL 没有常规的数据库重命名语句

不要依赖已废弃或版本相关的 `RENAME DATABASE`。需要重命名数据库时，推荐采用“备份 → 新建 → 迁移 → 验证 → 删除旧库”的流程：

```
mysqldump -u root -p --single-transaction --routines --triggers --events
olddb --result-file=olddb.sql

mysql -u root -p -e "CREATE DATABASE newdb DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;"
```

```
mysql -u root -p newdb --default-character-set=utf8mb4 -e "source
C:/path/to/olddb.sql"
```

这里的导出命令没有使用 `--databases`，因此备份文件不会强制使用旧数据库名；导入时明确指定 `newdb`，表对象会进入新数据库。
迁移完成并核对表数量、行数、触发器和程序连接后，再考虑删除旧库。重命名不是简单替换一个目录名，尤其不能绕过 MySQL 直接移动数据文件。

8. 删除数据库

8.1 删除语法

```
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name;
```

示例：

```
DROP DATABASE IF EXISTS school;
```

`IF EXISTS` 只是避免数据库不存在时的报错，并不会降低误删风险。

8.2 删除数据库的后果

> 不建议直接删除仍在使用的数据库。删除前必须先确认备份可用，并核对当前连接和数据库名。

`DROP DATABASE` 会删除该数据库下的业务对象及其数据。执行前至少应完成以下检查：

```
SELECT DATABASE();
SHOW CREATE DATABASE school;
SHOW TABLES FROM school;
```

删除是不可逆的 DDL 操作，不能用普通 `ROLLBACK` 恢复。除非事先有可用备份，否则不能依赖数据库自动找回。
还要区分三种看似相近、实际作用不同的操作：

操作	影响范围
`DROP DATABASE school;`	删除整个数据库及其中对象
`DROP TABLE student;`	只删除指定表
`DELETE FROM student;`	删除表中的记录，但保留表结构

如果只是清空数据并保留表结构，应使用 `DELETE` 或后续学习的 `TRUNCATE`，而不是删除数据库。
若当前数据库被删除，后续未带数据库名的操作可能提示 `No database selected`，需要重新执行 `USE`。

9. 数据库备份与恢复

9.1 备份保存的是当前状态

有一个容易产生误解的认识：“备份好像把历史上所有操作都备份了。”
更准确地说，`mysqldump` 生成的是**导出时刻的数据库结构、数据和部分对象定义**。备份文件里的 `CREATE`、`INSERT` 等语句只是重建当前状态的一种方式，并不等于保存了完整操作历史。若要追踪历史变更，通常还需要配合 MySQL 二进制日志等机制。
因此，备份至少应满足：

- 导出成功的退出状态。
- 包含所需要的表、数据、触发器、存储过程和事件。
- 定期做恢复演练。
- 备份文件与数据库字符集一致。

9.2 备份一个数据库

```
mysqldump -u root -p --single-transaction --routines --triggers --events --
databases school --result-file=school_backup.sql
```

关键选项：

- `--databases school`：在备份中加入创建数据库和选择数据库的语句，恢复时更容易还原同名数据库。
- `--single-transaction`：适合以 InnoDB 为主的数据库，尽量取得一致性快照。

- `--routines`、`--triggers`、`--events`：同时导出存储过程、函数、触发器和事件，实际可用性取决于权限及版本。
- `--result-file`：由 `mysqldump` 直接写入文件，可避免 PowerShell 重定向产生不合适的文本编码。
  如果只需要迁移到另一个数据库名，可以不加 `--databases`，先创建目标数据库，再把备份导入目标数据库。

9.3 恢复数据库

在 Bash 或 Windows 命令提示符中：

```
mysql -u root -p --default-character-set=utf8mb4 < school_backup.sql
```

PowerShell 不支持这种 `<` 输入重定向写法。可以在 PowerShell 中调用 `cmd`：

```
cmd /c "mysql -u root -p --default-character-set=utf8mb4 <
school_backup.sql"
```

也可以在已进入 `mysql>` 客户端后使用 `source`：

```
SOURCE C:/path/to/school_backup.sql;
```

使用 `--databases` 备份时，恢复命令会按备份中的数据库名创建或使用对应数据库；如果备份中没有数据库定义，需要事先创建目标数据库并执行 `USE`，或把目标数据库名写在 `mysql` 命令末尾。
恢复后必须重新执行以下检查：

```
SHOW DATABASES;
USE school;
SHOW TABLES;
SELECT COUNT(*) FROM student;
```

10. 综合实操与检查清单

10.1 从零创建并验证数据库

```sql
-- 查看已有数据库，避免重名
SHOW DATABASES;

-- 创建业务数据库
CREATE DATABASE IF NOT EXISTS school
    DEFAULT CHARACTER SET utf8mb4
    DEFAULT COLLATE utf8mb4_unicode_ci;

-- 查看定义并进入数据库
SHOW CREATE DATABASE school;
USE school;
SELECT DATABASE();

-- 验证数据库默认编码
SHOW VARIABLES LIKE 'character_set_database';
SHOW VARIABLES LIKE 'collation_database';
```

接着用学生表示例完成一次端到端验证：

```sql
CREATE TABLE student (
    id INT,
    name VARCHAR(32),
    gender VARCHAR(2)
);

INSERT INTO student (id, name, gender) VALUES
    (1, '张三', '男'),
    (2, '李四', '女'),
    (3, '王五', '男');

SELECT * FROM student;
```

修改数据库默认字符集后，再次执行 `SHOW CREATE DATABASE school;` 观察结果：

```sql
ALTER DATABASE school
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

最后模拟一次备份和恢复。只有在确认备份文件可以恢复、业务不再依赖旧库后，才可以删除测试数据库：

```sql
DROP DATABASE IF EXISTS school;
```

10.2 操作逻辑总结

操作	核心语句	关键注意点
查看数据库	`SHOW DATABASES;`	修改前先确认目标库是否存在
创建数据库	`CREATE DATABASE ...;`	可指定字符集和校验规则；不能指定存储引擎
查看定义	`SHOW CREATE DATABASE ...;`	核对默认字符集和校验规则
切换数据库	`USE db_name;`	只对当前连接会话生效
修改属性	`ALTER DATABASE ...;`	只改默认值，不自动转换已有表
删除数据库	`DROP DATABASE ...;`	高风险且不可滚动撤销，必须先备份
备份数据库	`mysqldump ...;`	保存当前逻辑状态，不是操作历史
恢复数据库	`mysql ... < backup.sql`	注意目标库、编码和对象完整性

10.3 常见错误

- **忘记选择数据库**：先 `USE db_name;`，或使用 `db_name.table_name`。
- **把数据库字符集当成存储引擎**：字符集决定编码，存储引擎在表级决定数据和索引的组织方式。
- **误以为修改数据库字符集会转换旧表**：已有表需要单独的 `ALTER TABLE ... CONVERT TO ...`。
- **认为备份包含所有历史操作**：逻辑备份重建的是导出时刻的当前状态。
- **直接删除数据库做“清理”**：删除前要检查当前库、表清单、依赖和可恢复备份。
- **在新版 MySQL 中继续使用旧 `utf8` 保存全部 Unicode**：新建库通常应使用 `utf8mb4`。